

第九讲

人工智能的计算基础

明玉瑞 Yurui Ming
yrming@gmail.com

声明 (Disclaimer)

本讲义在准备过程中由于时间所限，所用材料来源并未规范标示引用来源。所引材料仅用于教学所用，作者无意侵犯原著者之知识产权，所引材料之知识产权均归原著者所有；若原著者介意之，请联系作者更正及删除。

GPU—源起

- ▶ 图形处理器（Graphic Processing Unit或GPU）起源于早期的视频卡（也称为显示控制器），当时主要用作主处理器（CPU）和显示器（Monitor）之间的通道（pass-through）。视频卡主要包括视频移位器（Video Shifters）和视频地址生成器（Video Address Generators）两部分。视频卡将传入的数据流被转换为串行位图视频输出，例如亮度，颜色以及垂直和水平复合同步，然后将相应的像素信息在显示器中生成，包括同步每条连续行以及消隐间隔。早期GPU主要用于2D文字与图像的显示，结构简单，功能有限。
- ▶ 视频卡最初推出时，主要是针对企业市场或专业领域。后面的改进主要针对相关功能，提高芯片的晶体管密度，提升显示画面的分辨率，增加帧率、色彩等。同时随着消费级个人电脑的兴起，逐渐有新兴公司推出针对消费级市场的视频卡（通常称为图形适配器），典型如ATI公司在1985年推出的称为颜色模拟卡的图形适配器。虽然其以OEM方式发售，并不会直接面向终端消费者。但其靓丽的销售额还是引起了众多厂商的兴趣。

GPU—发展

- ▶ 由于图形显示是数据密集型应用，因此，为了显示的流畅性与功能的完整性，这些视频卡开始增加内存以及相关操作，如DMA等。同时，显卡最开始是用来显示文字与图像等，但一些工业与军事应用要求计算机能模拟较为复杂的3D环境，单靠CPU可能无法满足要求。将CPU的部分功能移到显卡上，即为显卡增加专属的处理核心，进行与图形图像相关的复杂指令操作，从而使CPU能专注于逻辑，是较为直观的想法。
- ▶ 但是，由于市场上众多公司的存在，尽管关于显示制定了一些标准，如VGA等，但关于图形处理，碎片化比较严重，为用户带来了一定程度的不便与困扰。1992年1月，Silicon Graphics公司（SGI）发布了OpenGL 1.0，这是一个用于2D和3D图形的多平台无关的应用程序编程接口（API）。OpenGL与微软后来发布的Direct X系列，成为主流的图形接口标准。他们均是与硬件无关的规范，不同的硬件厂商只要遵循这个规范就可以实现兼容。这大大促进了厂商之间的发展与优胜劣汰。

GPU—发展

- ▶ 特别需要提及的是，3Dfx公司的产品Voodoo graphics与VideoLogic的渲染技术，对推动显卡的发展有重要意义。特别需要指出的是，Rendition的产品Vérité V1000，提供了一个可编程的核心，开创了着色器编程的先河，为后来利用GPU进行通用计算埋下了伏笔。
- ▶ 但是由于一些市场因素，包括公司的自身创新，例如，英伟达（NVIDIA）通过提供基于三角形的多边形渲染，增加了Direct3D兼容性，因此，到90年代后期，特别在消费市场的主要玩家，基本剩下了NVIDIA，ATI与3Dfx。尽管一些公司如Matrox，Intel，在特定年份偶有爆款，但不少公司，要不销声匿迹，要不几经转折，被出售合并。例如，著名的3DLabs，先是被合并进Creative的SoC部门，后又卖给Intel；SGI在与NVIDIA和解后，后被并入NVIDIA。尽管Intel在显卡市场多有建树，例如，提出了AGP接口，但Intel专注于集成显卡，因此市场上，逐渐变成了ATI与NVIDIA两个玩家。



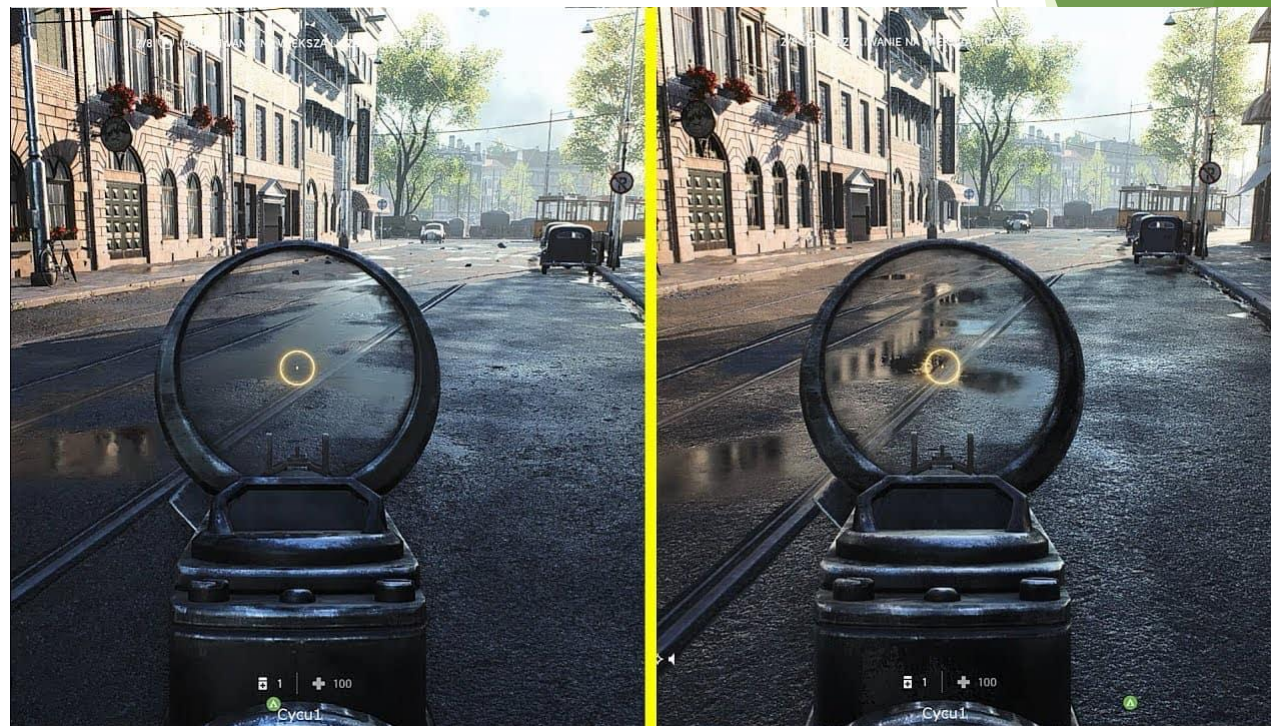
GPU—相争

- ▶ 英伟达在2000年发布的GeForce 256，标志着现代意义的GPU（图形处理器）的图形芯片的开始。通常在计算机图形学中，需要将3D对象和场景（及其相关光照）转换成渲染图像的2D表示，这通常要进行大量的浮点运算密集计算。以前，这种计算是由CPU承担的，而CPU在高负载的情况下容易造成瓶颈，但GeForce 256在硬件级别实现了转换和光照引擎（TnL或T&L），从而实现了更丰富的渲染细节。当然，这块显卡具有可编程着色器，并且支持DDR内存。



GPU—相争

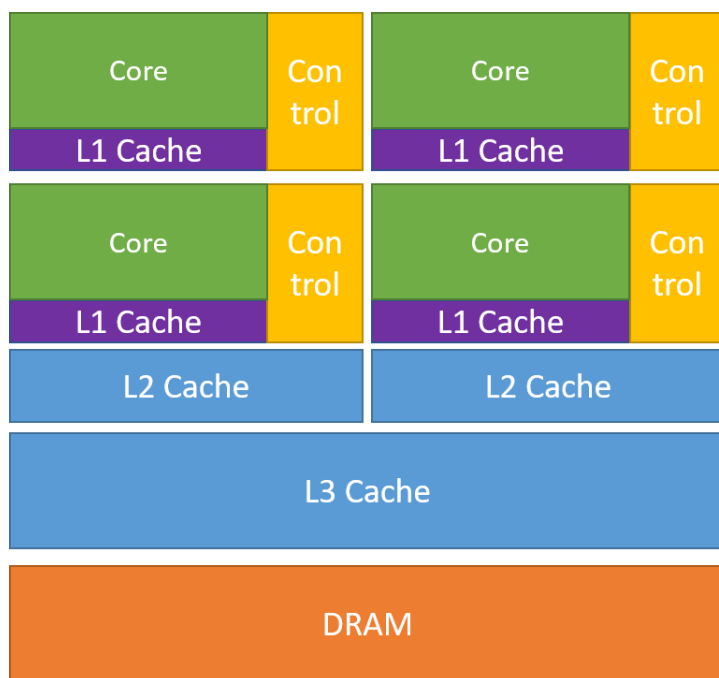
- ▶ 之后，NVIDIA与ATI的发展基本沿着提高工艺（流片的纳米尺度与晶体管集成度），提高显存，提高显卡的时钟频率，提高流处理单元的数量，提高显卡计算核心与显存的内存字长，提高计算核心与显存的吞吐带宽，提高显示图像的分辨率与刷新速率，提高硬件渲染功能等方面进行。



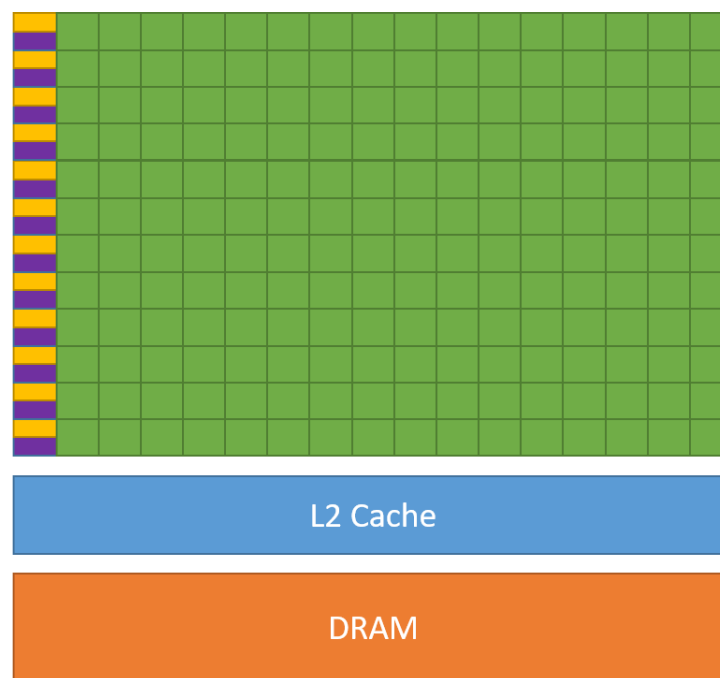
- ▶ 右图显示了硬件支持的实时光线追踪（右）与无光线追踪（左）的对比。由于光线追踪需要消耗大量的硬件计算资源，这潜在地促使了人们思考如何在GPU的计算资源空闲时，加以利用做其它的事情。

GPGPU

- 实际上，在显卡的发展过程中，更多的功能与更大的灵活性被逐渐加到可编程着色器里面；同时，基于渲染管线的流计算特点，着眼于GPU高并发、低中断、计算密集的特征，一些学者逐步考虑是否可以将GPU用到特定的通用计算里面。



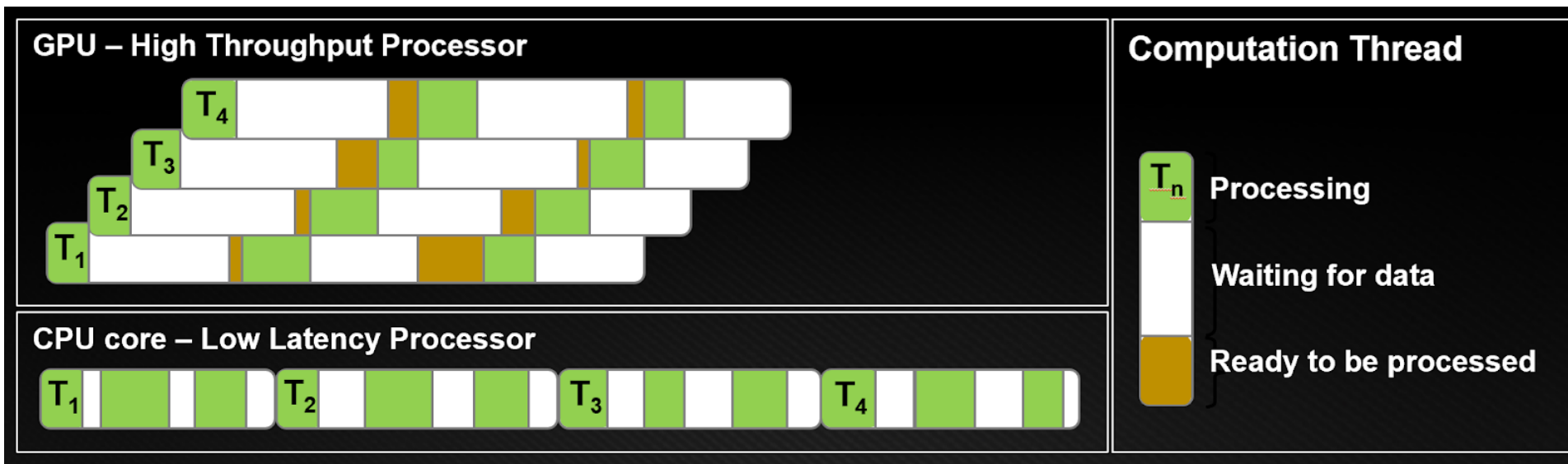
CPU



GPU

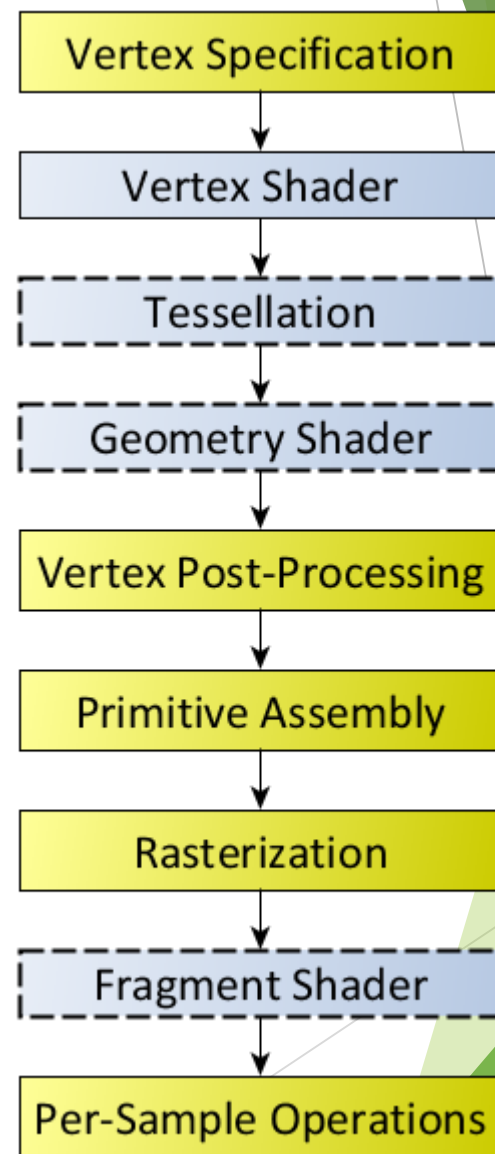
GPGPU

- 但是，将一般的计算模型或计算过程，适配到显卡的渲染管线，并非轻而易举的事情。首先，在设计上，CPU与GPU就有着较大的差异。CPU的线程通常是低并发重量级的线程，例如，当一个线程在等待数据时，通常会被中断，从而将CPU时间片让渡给资源已经就绪的线程；而GPU线程是轻量级高并发线程簇，一般开始后不会被中断，一直到执行结束；等待数据时，也只是空转。



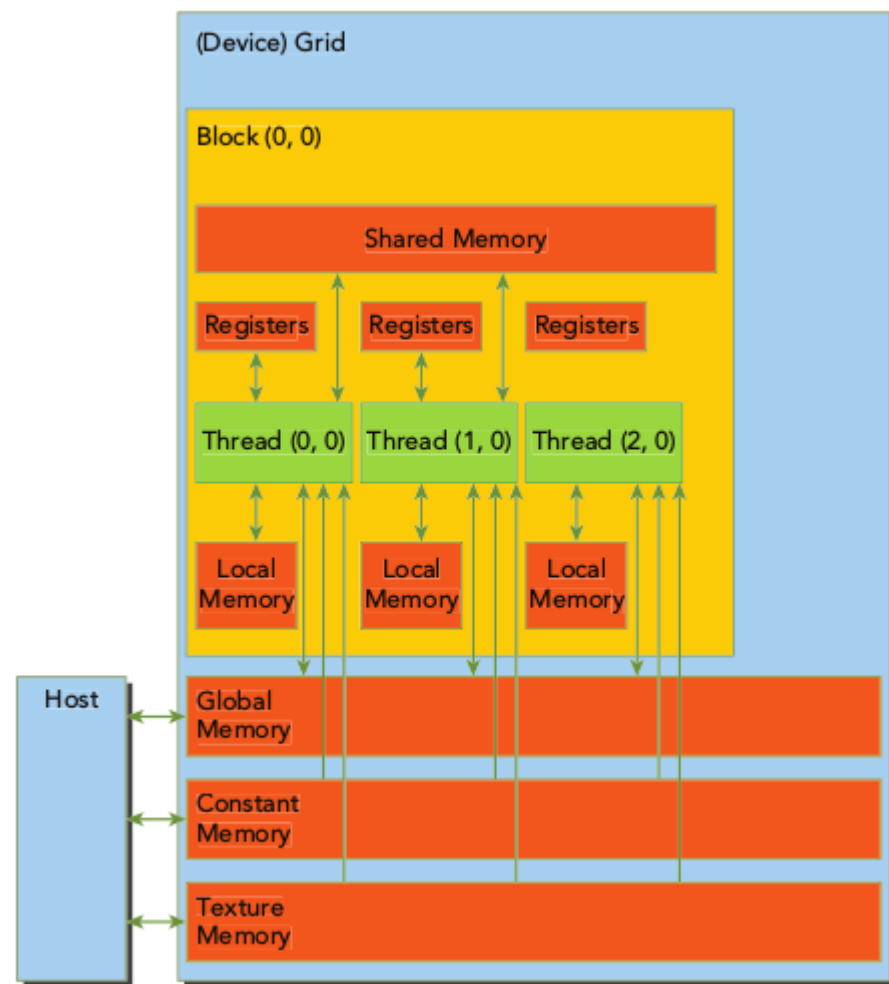
GPGPU

- ▶ 同时，为了适配渲染管线，例如，着色器被显式地分为定点着色器（Vertex Shader）、片元或片段着色器（Fragment Shader），因此，显卡的内存也被分为多种类型，例如常量内存（Constant Memory），顶点内存（Vertex Memory）、片元内存（Fragment Memory）、纹理内存（Texture Memory）等。这显然是将通用计算适配到渲染过程的又一障碍。
- ▶ 利用GPU进行通用计算的初探落地在如何利用渲染管线或其一部分里的这些可编程着色器。但最开始，对这些可编程着色器的编程用一种类似汇编语言的方式进行，同时根据所要交互的内存，添加一定的约束；尽管后来一些标准如OpenGL、DirectX添加一些扩展以推动通用计算方面的工作，但毫无疑问，这要求用户对显卡的工作过程有较深的了解，这些无疑都是将GPU用诸于通用计算的障碍。



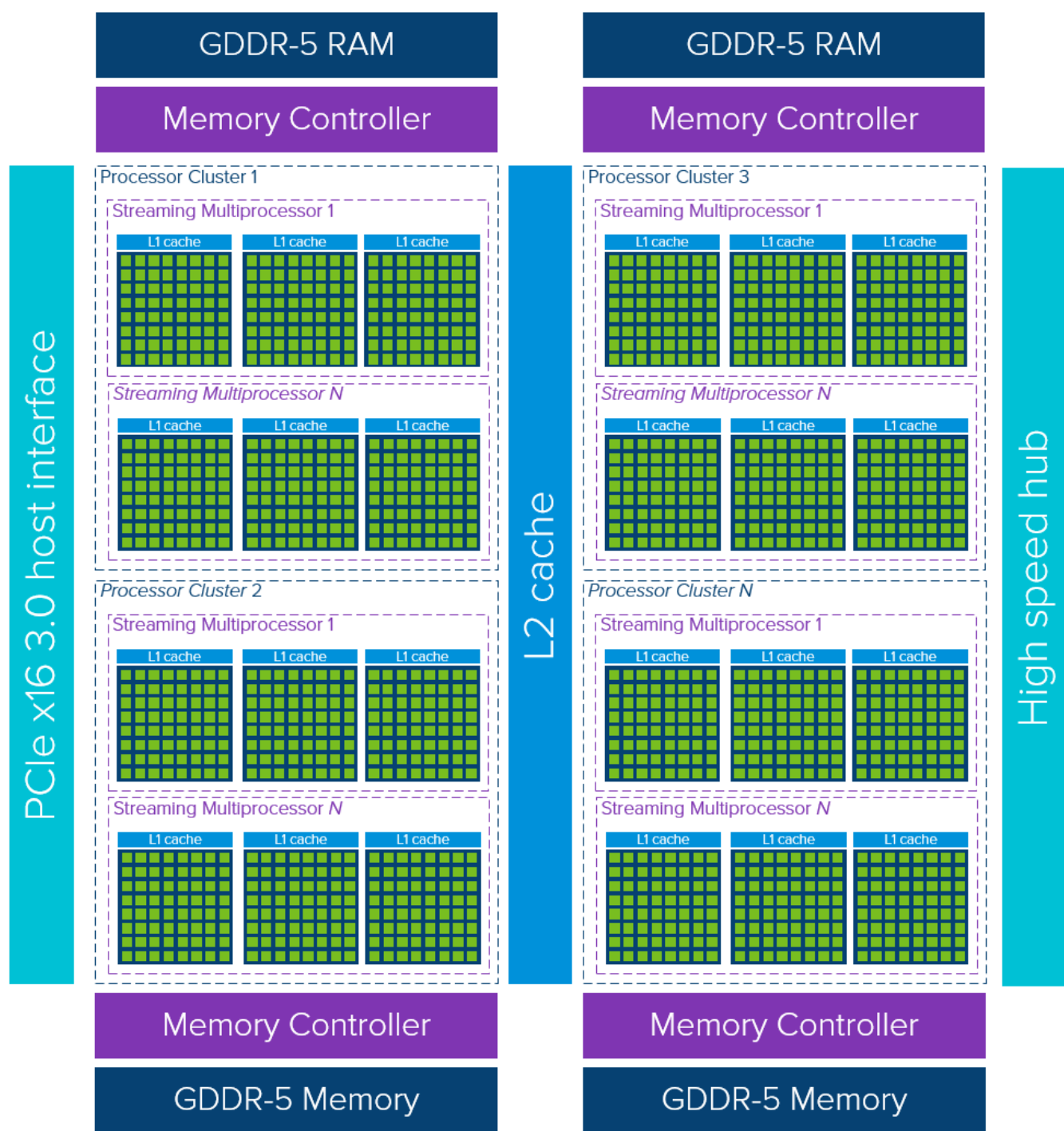
GPGPU

- ▶ 实际上，除了来自业界和用户方面对显卡承担通用计算的期盼，GPU厂商也面临着另外的窘境。为了满足特定游戏场景或模拟环境的需求，GPU在后来的渲染管线中又加了可编程几何着色器（Geometry Shader）；并且，随着越来越多的硬件功能的加入，一些硬件电路在顶点着色器、几何着色器、片元着色器等之间是重复的。为了减少设计的复杂性和提高硬件电路的利用率，设计一种新的统一的着色器模型，已经成了势在必行的工作。
- ▶ 当时从逻辑上将显存区分为顶点内存、常量内存、纹理内存等，在一定程度上是为了配合不同类型的着色器使用。当考虑将着色器进行统一时，则逻辑上对内存的分类也显得不必要。
- ▶ 统一的着色器模型与统一的内存模型对将GPU从单纯渲染用于渲染+计算具有重要的意义。该模型大大降低了在软件层进行抽象，映射到硬件层面实现的难度，为相关工具软件的开发奠定了基础。



GPGPU

- 下面图展示了现代的GPU架构，以NVIDIA的统一计算设备模型（Compute Unified Device Architecture或CUDA）为例。从硬件层面上看，渲染管线的各个阶段已越来越不明显，这正是统一着色器模型以及统一内存模型带来的变革：



LLVM

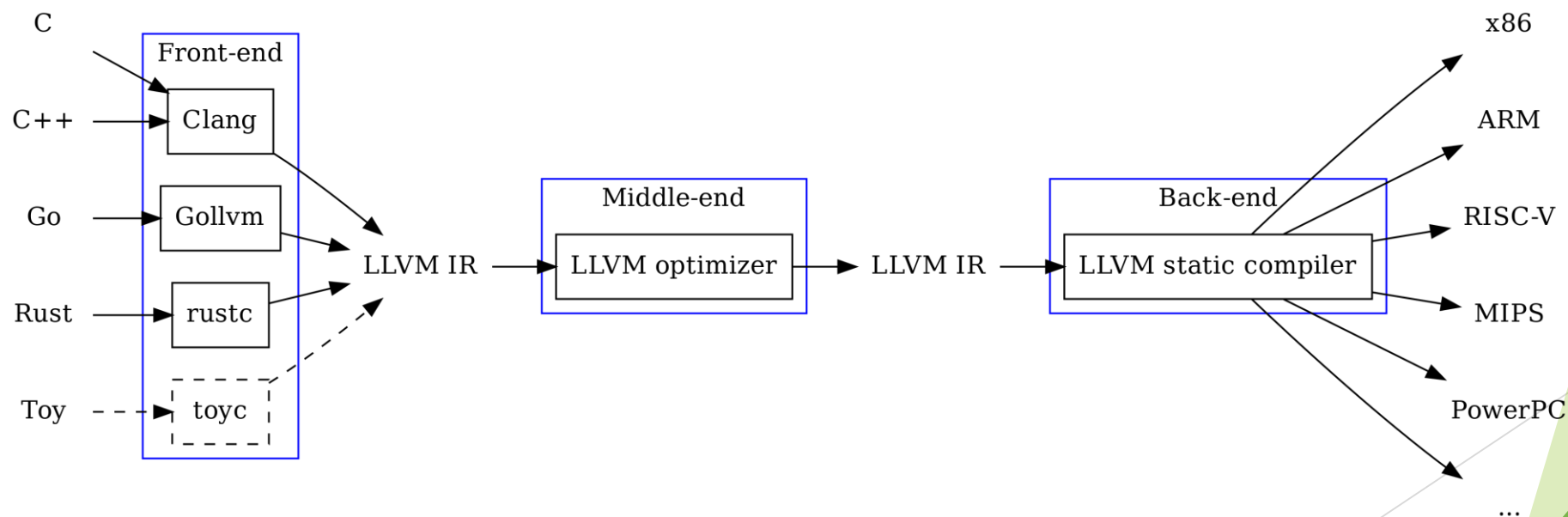
- ▶ 如前所述，显卡渲染管线中的顶点着色器与片元着色器起初都是固定功能的。为了满足日益复杂的显示与渲染的需求，后来发行的显卡将这些着色器以可编程的方式呈现，方便公司与用户定制自己的渲染过程。为适配这些，OpenGL提供了称为GLSL的着色器编程语言，DirectX提供了称为HLSL的着色器编程语言。OpenGL与DirectX并没有限制这些语言只用于渲染，因此，最初的通用计算探索如何利用这些着色器编程语言，将特定类型的计算映射为渲染过程进行计算。但这通常依赖于精巧的设计，且费时费力。
- ▶ 尽管后来有一些语言或库，例如cg等，降低了编程难度，但从用户的角度，还是具有很高的门槛。但其中的原因，并不能全部归咎于显卡制造公司。通常，将C语言等高级语言转变成低级机器语言，依赖于编译器、链接器等工具。在开源工具例如GCC上进行扩展，往往是许多厂商的首选。但随着CPU技术的进步，指令集的添加，一些GCC功能的增强可能通过补丁（Patch）的方式进行，这导致GCC越来越庞大。这种尾大不掉的状态，导致修改GCC去适配新的硬件架构非常困难，也即厂商在针对显卡进行通用计算的软件生态建设上，也举步维艰。

LLVM

- ▶ LLVM编译框架的出现，为突破这个困局提供了新的希望。
- ▶ 故事要从苹果公司（Apple）讲起。Apple最初是使用GCC作为官方的编译器的，但GCC对Apple的Objective-C语言的新增特性支持并不好，Apple不得不将GCC分出一支进行开发。结果就是Apple在自己的分支上开发Object-C语言的特性，然后同步到主分支；同时将主分支上的特性同步到自己的分支。但这样做有一个问题，即Apple尽管费时费力，但其编译器版本常常落后于GCC的官方版本。
- ▶ 另一方面，GCC的代码耦合度太高，不好独立，而且越是后期的版本，代码质量可能越差。更有甚者，后来的“Exceptions to GNU Licenses”限制了GCC的定制开发。因此，寻找一个高效的、模块化的、协议更放松的开源替代品，成为Apple的首选。
- ▶ 后来，毕业于伊利诺伊大学厄巴纳香槟分校，专门做编译器优化研究的Chris Lattner被招至Apple麾下，主导新的编译框架的开发。他的硕士毕业论文就提出了一套完整的在编译时、链接时、运行时甚至是在闲置时优化程序的编译思想，奠定了LLVM的基础。LLVM在他念博士时更加成熟，使用GCC作为前端来对用户程序进行语义分析产生IF（Intermediate Format），然后LLVM使用分析结果完成代码优化和生成。

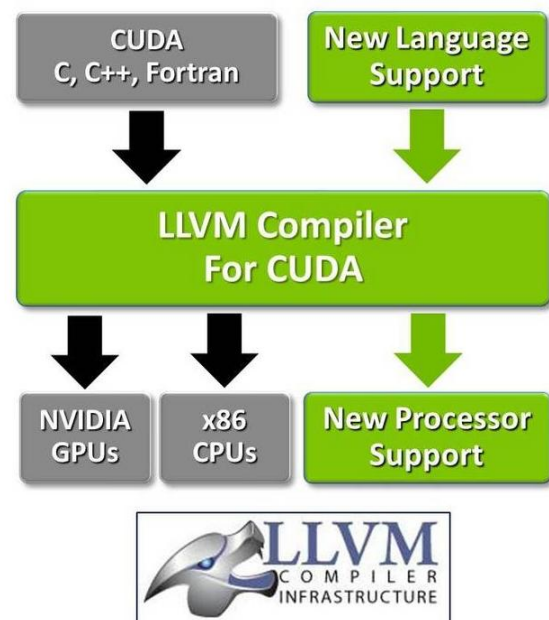
LLVM

- ▶ LLVM最初设计时，着手于做优化方面的工作，所以需要搭建一套虚拟机，所以当时这个的全名叫Low Level Virtual Machine。后来因为要变成更宏伟目标的编译器框架，官方就放弃了这个称呼，但LLVM的简称还是保留下来了。



LLVM

- ▶ 因为LLVM只是一个编译器框架，所以还需要一个前端来支撑整个系统，所以Apple又组织研发了Clang，作为整个编译器的前端，将源文件编译成中间IR，进行深度优化后再进行接下来的工作。测试表明，此举在代码优化、速度和敏捷性上均优于GCC。实际上，刚进入Apple工作时，Chris Lattner首先在OpenGL小组做代码优化，把LLVM运行时的编译架在OpenGL栈上，这样OpenGL栈能够产出更高效率的图形代码。总之，这些大大增强了其它公司基于LLVM进行构建工具研发的信心。
- ▶ 实际上，CUDA最开始是从硬件层面来讲的，在一定程度上是为了适配DirectX 10的一些要求。对这些着色单元从计算角度的使用，仍需借助如DirectX的相关API。但LLVM的成功，使NVIDIA公司看到直接将C等代码编译为在CUDA架构上的机器码的可行性。在具有CUDA架构的显卡发行约2年之后，NVIDIA于2006年推出了CUDA开发工具包，包括相应的SDK与Toolkit，这大大降低了利用GPU进行通用计算程序编写的难度。

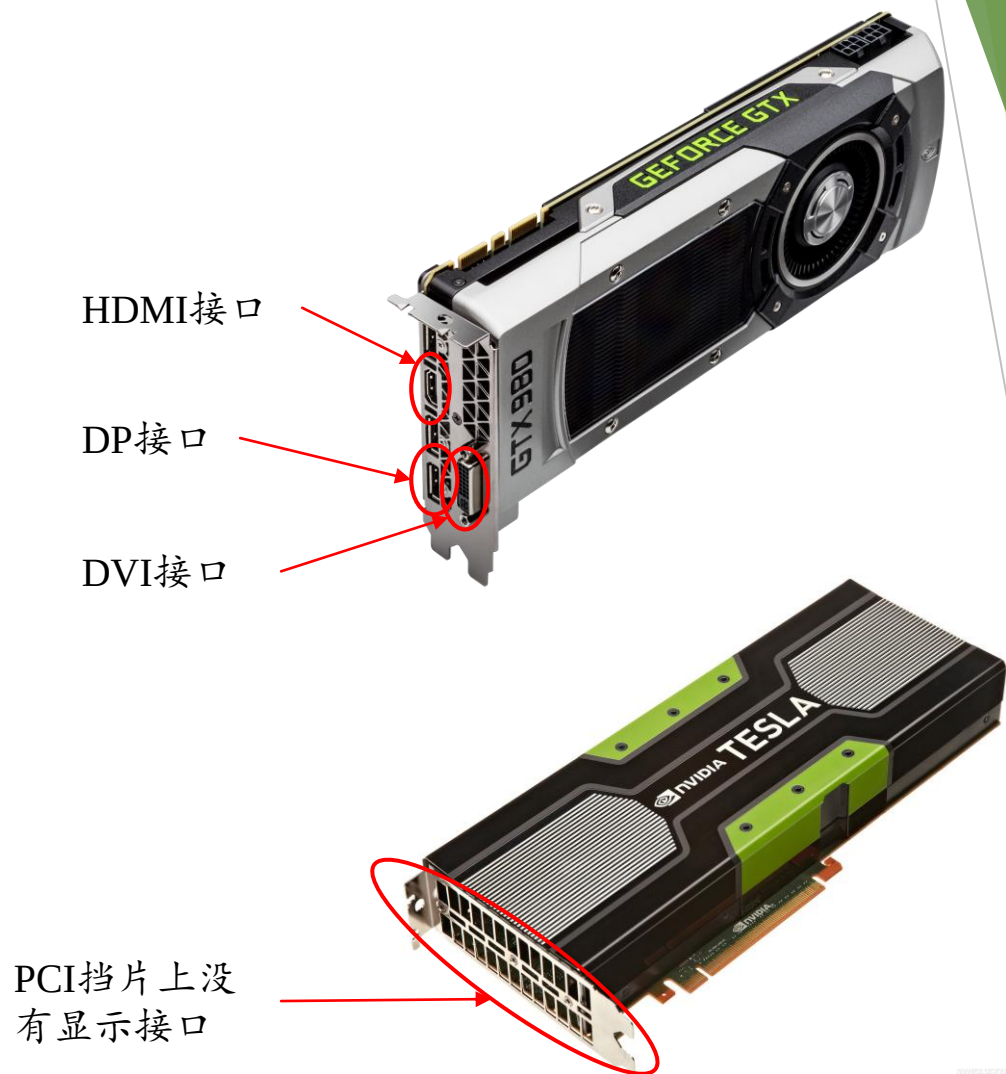


CUDA

- ▶ 随着CUDA软件开发工具的推出，基于GPU的科学计算大规模开展起来。但基于GPU训练深度神经网络，直到2012的标志性工作AlexNet才引起人们的注意，原因是多方面的：
 - ▶ 在本世纪初SVM因其扎实的数学基础与清晰的应用解释，是人们研究的热点；相对的，神经网络并不受人们的重视。除了Geoffrey Hinton在多伦多大学独立支撑外，并没有太多的实验室进行这方面的研究。
 - ▶ 对于深度神经网络，Hinton关于逐层训练的工作是把双刃剑。他在提示逐层训练然后微调的可行性的同时，也可能暗示着训练的困难性，人们可能认为算力仍是问题。
 - ▶ 利用GPU进行端到端训练没有先例，而对CUDA的掌握，有一定的技术门槛。在没有可供参考实现的情况下进行从无到有的开发，存在着很大的挑战。
- ▶ 但是当Alex Krizhevsky展示了可以在两块消费级显卡上成功训练卷积网络并在ImageNet上取得当期最好的结果时，立刻引起了学术界与工业界巨大的兴趣。谷歌曾在2010年利用16000个CPU来训练一个神经网络识别互联网上猫的图片，这个工作在AlexNet后，于当年在48块显卡上重做。这个工作也导致了cuDNN库的诞生，即NVIDIA以官方的姿态，基于CUDA加持深度神经网络的训练。

CUDA

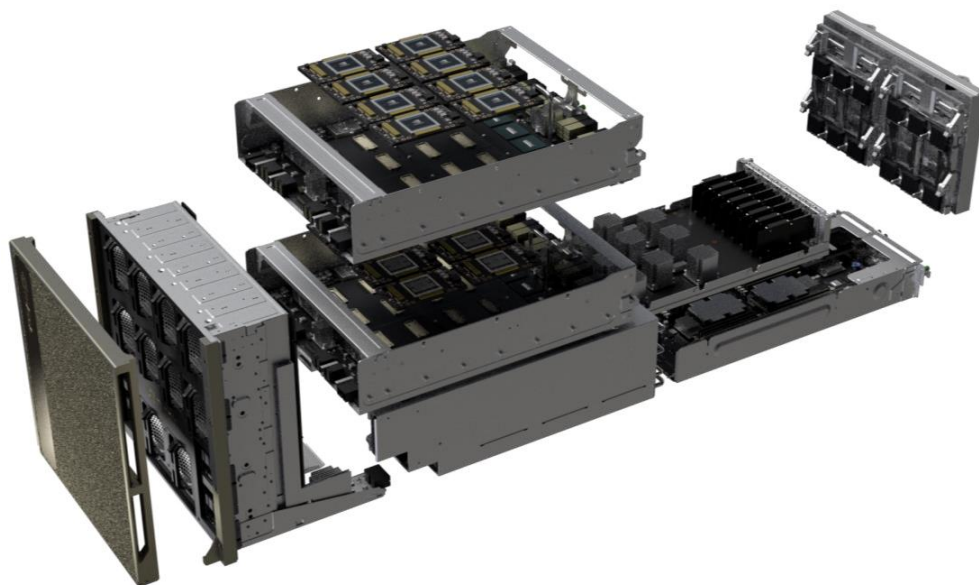
- ▶ 随着人工智能进一步的发展，NVIDIA围绕深度学习在硬件与软件布局的越来越完善，更多的企业依赖于NVIDIA提供的基础设施来进行深度学习算法的研究与开发。往往这些基础设施的部署，会以数据中心的方式建设，以云服务的形式提供给终端用户。此时，显卡关于渲染的功能并非必要，因此，NVIDIA又推出了一系列仅有计算功能的显卡，称为计算卡。这些计算卡基本都具有较大的体积，较多的显存与流计算单元，较高的功耗，一定程度上支持多显卡互联以聚合更大的显存。同时，NVIDIA也推出了更多的库，如TensorRT等，支持深度模型的部署与推理。



CUDA

- ▶ 总之，NVIDIA在深度学习方面的耕耘与建树不仅推动了AI的发展，也使公司的业务蒸蒸日上，从显卡时代NVIDIA与ATI两家争雄的局面，变成了NVIDIA一家独大。当然，NVIDIA的战略眼光，其它厂家的紧追不舍，驱使NVIDIA继续在从基础设施，特别是算力角度服务人工智能的发展上，贡献着自己的能力并收获着市场份额。但毫无疑问的是，GPU从渲染到通用计算，以及基于GPU异于CPU的特点，有力保证了神经网络在算力上的需求，推动了以深度学习技术为代表的人工智能的发展。

- ▶ 右图展示了NVIDIA的DGX-2计算系统，服务于大规模的AI项目。

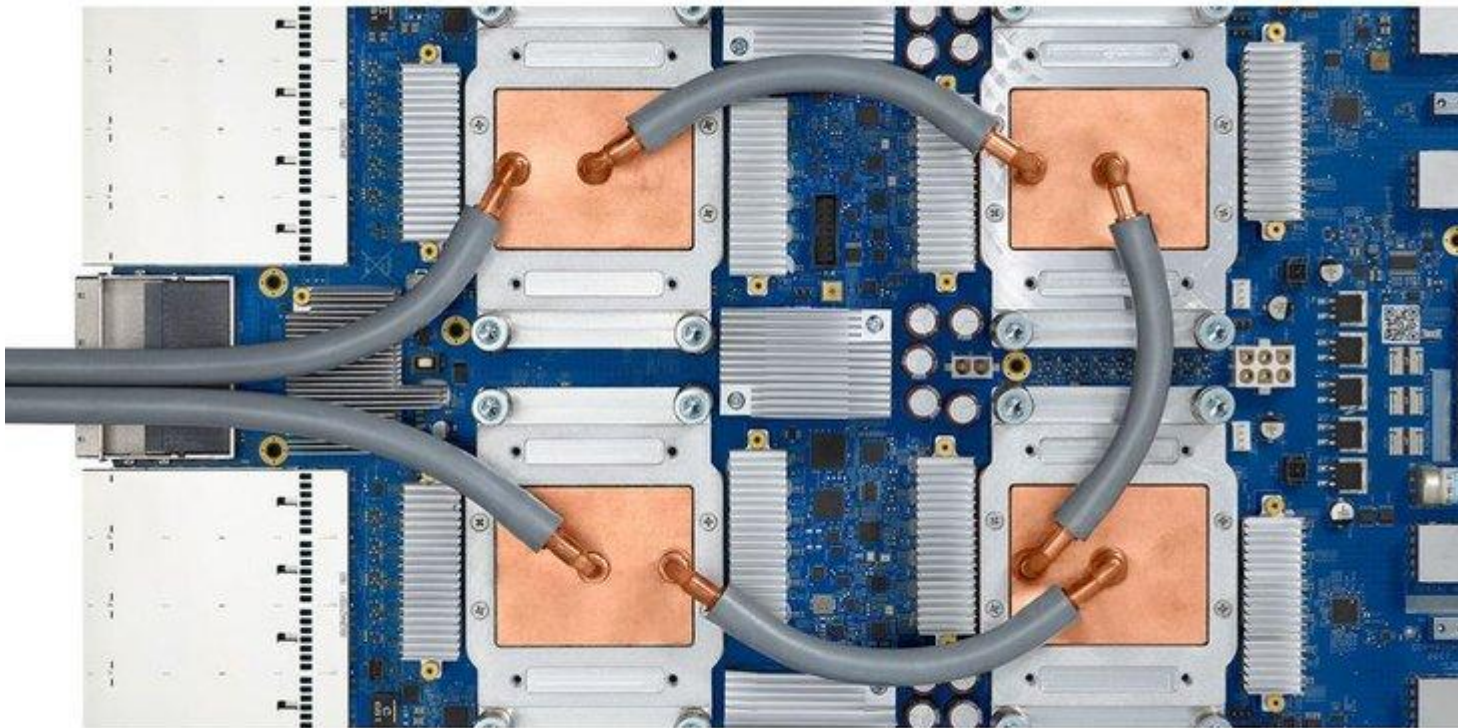


TPU

- ▶ 由于AI的持续火热，导致一些公司在硬件资源上的投入越来越大，典型的如谷歌（Google）公司。例如，谷歌在2013年就意识到AI导致的快速增长的计算需求，可能导致数据中心的数量需要翻番。当然，谷歌的动机还包括以下两点考量：
 - ▶ 随着AI在更大范围内普及，作为基础设施的GPU可能会成为瓶颈，这会潜在地导致芯片短缺，间接提高了模型的运行成本。且这个趋势有可能随着AI的爆发呈现恶化的趋势；
 - ▶ GPU最初设计是为了渲染，进行计算一方面是基于一般情况下，渲染任务不足导致的计算力的富余；同时，虽然GPU本身在一定程度上适合数据密集型的计算，但这并不是一开始就是为了神经网络的训练与推理量身定做，因此，效率并不算高。
- ▶ 以上等因素促成了谷歌公司为神经网络研发与定制专用芯片（ASIC），并且深度适配谷歌的深度学习框架TensorFlow，称为张量处理器（Tensor Processing Unit或TPU）。该处理器采用了28nm工艺制造，主频700MHz，功耗40W。为了提高效率，Google采用量化的技术进行整数运算，来避免传统的浮点数运算。同时，谷歌为GPU设计了称为矩阵处理器（MXU）的计算单元，其采用称为脉动阵列（Systolic Array）的架构，以适配神经网络的计算过程。

TPU

- ▶ 虽然谷歌选择复杂指令集(CISC)作为TPU指令集的基础，但由于TPU设计为一个单线程芯片，不需要考虑缓存、分支预测、多道处理等问题。CISC并不会大幅增加设计难度，相反，单CISC指令在进行复杂的计算任务的同时，可以实现较好的性能功耗比。
- ▶ 下图给出了TPU的图示。由于谷歌在其广泛的产品中用到了人工智能，因此，TPU除了满足谷歌公司自己的需求外，也激发了其它公司研发AI专有加速芯片的热情。同时，谷歌数据中心通过Colab方式，提供了TPU的计算能力给研究者与实践者使用，也有力地推动了基于神经网络技术的AI的发展。



TPU

- ▶ 虽然实施TPU的想法较早地在谷歌被提出，但从真正着手到第一版TPU在数据中心落地，谷歌大约用了15个月时间。这似乎给人一种假象，即制造AI专有加速芯片是一件容易的事情。但实际上，谷歌有好多得天独厚的条件：
 - ▶ 谷歌在硬件领域有较好的资源积累。谷歌庞大的服务内容需要大规模数据机房等硬件基础设施的支撑，谷歌在利用专有芯片在数据交换路由方面有丰富的经验，因此，在利用硬件做专有领域的加速方面，谷歌有着可以借鉴的实践资源。
 - ▶ 谷歌在软件方面有着丰富的经验。基于支撑自身业务的需求，谷歌在软件研发方面有着长期的巨大的投入，从程序语言、编译工具、应用软件三个层面，谷歌都有深厚的积累。如Go语言，Bazel构建工具、TensorFlow神经网络计算库等，均展示了谷歌强大的技术实力。
 - ▶ 谷歌在学界与业界有着广泛的影响。TPU由著名计算机科学家大卫·帕特森（David Patterson）领衔设计，他在算机体系结构与工程方面有着极为丰富的经验，是精简指令系统的两代设计者（MIPS与RISC-V），其对不同指令集下的系统效能有着深刻的认识，有着丰富的处理器芯片设计经验，且在项目成功的人力资源方面亦有着很高的威望。另外，作为LLVM的开创者，Chris Lattner也在谷歌大脑从事与TensorFlow相关的开发工作，再加上众多的优秀研发人员，这对TensorFlow适配TPU的工作方式，也提供了很好的背书。

AI加速芯片

- ▶ 显然通常公司并没有谷歌的技术实力与资源，但AI巨大的应用场景导致了提供AI算力芯片的潜在蓝海，因此，无论国内还是国外，许多公司都涌入这一赛道，进行相关产品设计与应用，特别是垂直领域的应用。
- ▶ 同时，NVIDIA的GPU+CUDA的方式又在行业树立了很高的技术与商业壁垒，特别对于初创公司，一个好的方向是在某个细分领域，针对特定应用，语音或视频，对基于神经网络的特定算法提供算力支持。并且绝大多数情况下，是在模型的部署应用阶段，即首先基于GPU训练好模型，然后再用AI加速部署在终端的模型的推断，这降低了产业化的难度。
- ▶ 一般的AI加速芯片或AI芯片由两种形式：
 - ▶ 一种是AI加速度卡，直接将AI加速核心固化做成芯片，作为系统外设，通过PCIE的接口挂到主机箱上。对于云端设备来讲，通常以此种方式进行，来增强系统提供AI服务的能力。且该方式与之前的系统设计兼容。
 - ▶ 另外一种形式是将AI加速核心作为主处理器或主控制器的协处理器，以模块的方式挂载到CPU的总线上。对于边缘端来讲，多采用此种方式。因为边缘设备往往不具备像PCIE一样的高速外围接口，且对成本比较敏感。作为协处理器直接挂载到CPU总线，既能满足速率的要求，一次性流片时又可节省成本。

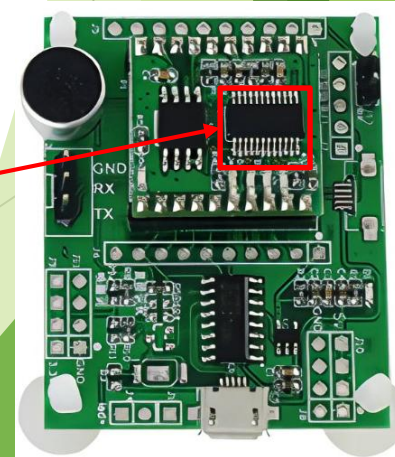
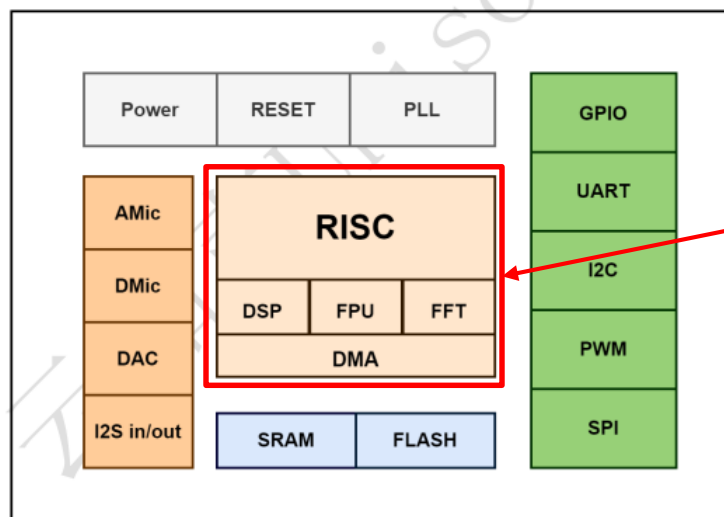
AI加速芯片

► 对于AI加速卡，通常可以有三种方式：

- 一种是神经网络处理器（NPU），典型例子是TPU。TPU从设计之初便着眼于较为完整的软件与硬件支持，能适应较为丰富的基于神经网络的应用场景，功能较为丰富，性能较为突出。但NPU的开发需要较长的时间周期，对公司在各个方面的技术实力有较高的要求，考虑流片等因素，短期成本较高，风险也较大。
- 一种是基于FPGA的加速器，典型例子是一些数据中心的外挂加速卡。其既可以在软件的支持下，作为独立的加速器运行，又可以单纯地将一些CPU需要的计算，下载（offload）到FPGA上执行。由于FPGA的运算可以通过软件配置的方式进行，因此不存在流片失败风险，但由于硬件功能通过软件配置方式实现，因此要求具有软件与硬件知识与技能兼备的工程师，基于Verilog HDL或VHDL等语言进行硬件功能设计。由于软件设计进行硬件综合时，不可能将FPGA所有资源用尽，因此，这种方案可能长期成本较高。
- 一种是基于ASIC的加速器，一些外挂加速卡通常也会以这种方式实现。但ASIC一般是用FPGA作概念验证，在证明无误后，然后再进行设计与流片。但是ASIC与NPU相比，原则上ASIC仅对特定应用进行加速，因为完整功能的NPU设计类似于CPU设计，大大超出了一般公司的技术能力。由于仅为特定应用进行加速，因此，对适配以ASIC为加速卡的系统，可能不要求软件系统有很高的复杂度，仅仅对需要加速的应用能适配即可。

AI加速芯片

- ▶ 对于协处理器，一般是基于ASIC的简化的神经网络处理器（NPU），直接挂到CPU或MCU总线上。通常，这里的总线均指高速总线。由于终端设备功能均比较特定，如亦或处理视频数据，亦或处理语音数据，但一般不会兼而有之，因此，其软件功能也不要求多复杂，能适配神经网络计算加速的功能即可。
- ▶ 另外一种形式是针对终端设备，一些厂商可能将整个CPU或MCU做成只针对特定任务的芯片，例如语音识别功能。由于任务单一，整个CPU或MCU的逻辑计算单元主要用来做神经网络的推理，因此在模型较小的情况下，性能尚可，但原则上这里并没有实质的加速。



国内现状

- ▶ 国内最开始做AI加速卡，最开始是从GPU开始的。国内厂商开始做GPU，也是从传统的2D/3D图形渲染GPU开始的。但是由于NVIDIA与ATI已经形成很高的技术壁垒，因此除了在军工与某些领域，国内厂商在消费市场并没有太大的影响力。近些年由于国外对国内的技术封锁，在国产化的过程中，一些GPU厂商配合国产CPU开始做一些生态工作，如景嘉微，其JM9系列与飞腾系列CPU进行系统集成，整体兼容性较好。
- ▶ 随着人工智能近几年如火如荼地开展，国内厂商开始在GPU高性能计算方面进行探索。相对图形渲染，纯计算功能的GPU在驱动软件、计算软件等方面相对较为简单，因此，有不少厂商在原有显卡芯片的技术的基础了，推出了相应的产品。如芯原股份，其Vivantan NPU IP系列，可提供相应的神经网络计算的硬件加速功能。
- ▶ 同时，为了适应市场的趋势，许多初创厂商进入该赛道，产品侧重于与AI相关的处理器芯片，通常以AI加速卡为主。一些成立较早的公司，如寒武纪，已形成了较为完整的产品线；另外一些公司，如海光信息，壁仞科技、摩尔线程、芯动科技等或侧重于服务器端（云端），或侧重于客户机（终端）的AI性能加速。
- ▶ 另外，一些成熟的芯片设计厂商，如海思科技、紫光展锐等，也设计了相关NPU芯片，以增强手机的智能功能的表现。

国内现状

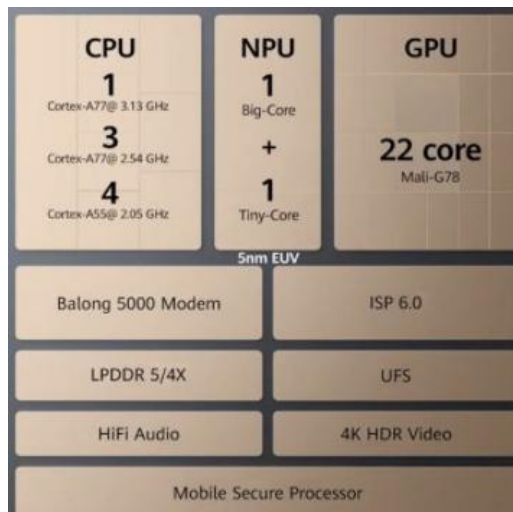
右图展示了一些公司的产品。



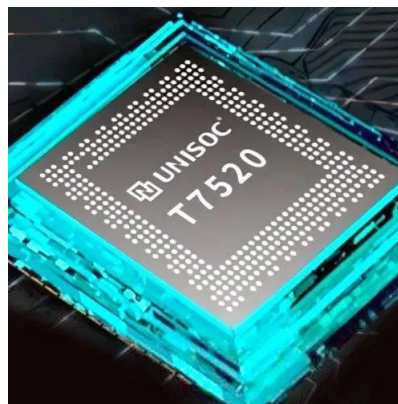
寒武纪思元370
智能加速卡



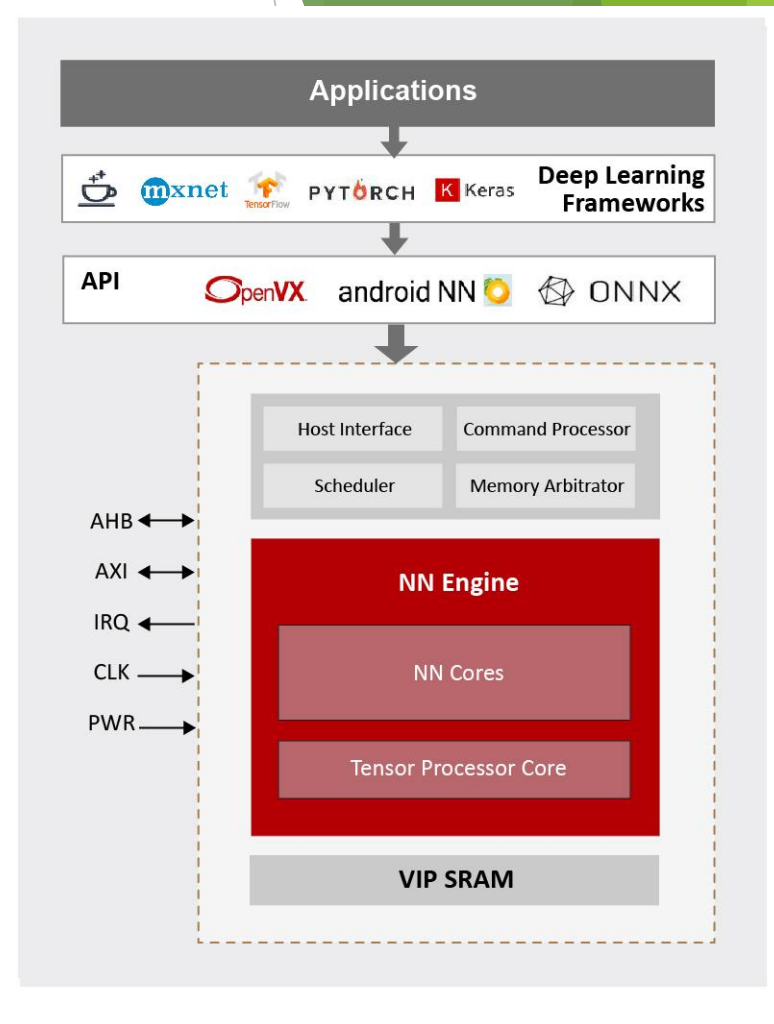
摩尔线程MTT
S50 显卡



海思麒麟9000E
系列处理器



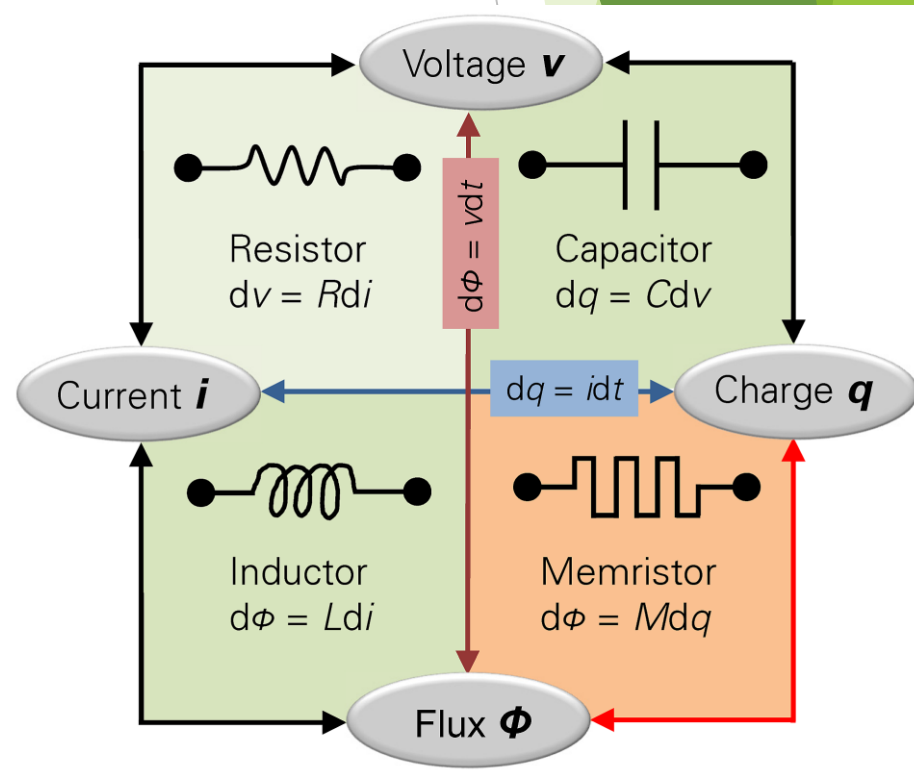
紫光展锐虎贲
T7520处理器



芯原股份
VIP9000Pico架构

未来趋势

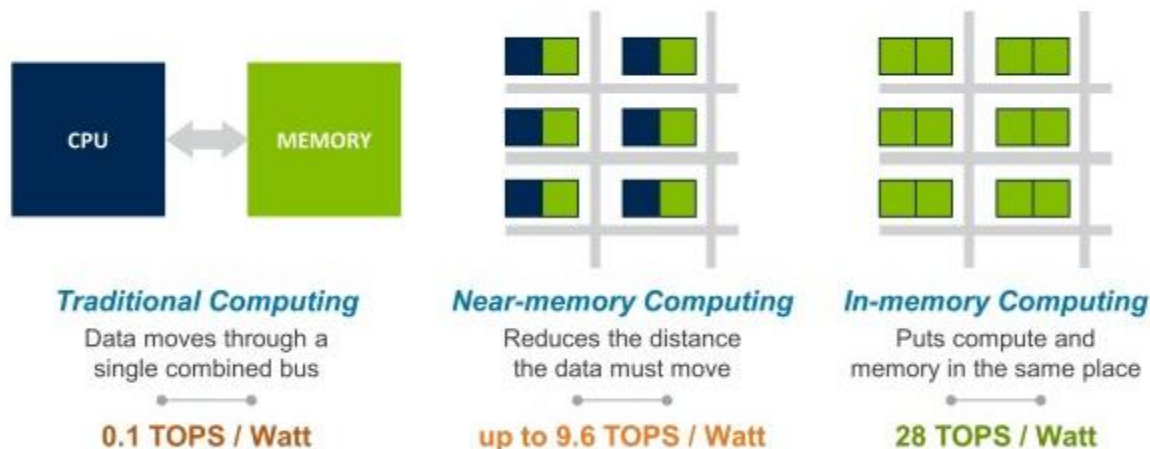
- 无论是以GPU的计算功能还是单纯的加速卡加速，神经网络模型的应用基本比较偏向固定的范式。通常是先训练模型，在精度满足要求之后，进行模型的部署。如果希望模型持续改进，则通常是在收集更多高质量之后，重新训练与重新部署。并且神经网络的结构依然要靠软件来实现，硬件主要用来做计算与加速。
- 但相对生物智能来说，生物的学习总是在持续进行的，并且学习的基础与生物结构密切相关。例如，特定学习能力的提高在一定程度上伴随着突触的生长，且并没有显式的学习阶段与实践阶段。从硬件的角度探索基于神经网络的持续学习能力，是十分有必要的。
- 忆阻器由美国加州大学伯克利分校的华裔科学家蔡少棠教授在1971年提出。忆阻器代表着电荷与磁通量之间的关系，其电阻会随着通过的电流而改变，即这样的组件会「记住」之前的电流量，因此被称为忆阻器。其后，由HP公司在2008年研究阻抗存储器（PRAM）时，发明了较为实用的忆阻元件。然后，学术界与业界便对忆阻器展开了如火如荼的研究。



未来趋势

- 对于神经网络，即使是人工神经网络，也可以从多个角度解释。例如，对卷积神经网络而言，既可以从滤波器角度考虑，即从计算的角度考虑，认为神经网络对输入的处理是基于计算的卷积操作；也可以从模板匹配的角度考虑，即从记忆的角度考虑，神经网络包括了处理特定问题的知识。

- 实际上，这种将记忆与计算融为一体的方式，不仅可能是生物智能所采用的方式，而且是计算机科学研究的一个重要方向。下图展示了不同架构下系统的吞吐量，可以看出存算一体架构的优越性：



未来趋势

- ▶ 突触的可塑性在一定程度上表现为突触对局部电场的调制，且这种调制是非易失的。这对人工神经网络来说，即表现为权重。而这种特性可以很容易映射到忆阻器的电阻（或电导）。而神经突触的可塑性对应的是学习过程，因此，藉由忆阻器，可以构建存算一体的新型神经网络芯片，并实现持续学习或增量学习的功能。而这点，我国走在了世界的前列，并将对AI的未来发展产生一定的影响。

